Towards Unsupervised Time-Series Anomaly Detection for Virtual Cloud Networks

Zixuan Ma, Chen Li, Kun Zhang, and Bibo Tu

***Abstract*—Virtual cloud network (VCN) is a fundamental cloud resource for endpoints (VMs or containers) to communicate with each other and with the outside. Anomaly detection, a key security approach for VCNs, faces serious challenges: 1) Current feature models are difficult to apply to VCNs with significant differences from traditional networks. 2) Current anomaly detection models lack the adaptability to learn multiple normal patterns simultaneously. The need to train a dedicated model for each endpoint causes serious scalability problems in VCNs. 3) Current anomaly detection models have difficulty addressing the complex temporal dependency and non-stationarity of VCNs. To address these challenges, we propose a new multilevel feature model MFM and a new unsupervised time-series anomaly detection model GTGmVAE. By combining the basic features with the topology features specifically designed for VCNs, MFM effectively characterizes the patterns of VCNs. GTGmVAE combines the new local-global feature extractor with the latent space following a Gaussian mixture distribution to achieve the strong adaptability to learn multiple normal patterns simultaneously, and achieves the strong temporal modeling capability to effectively address the complex temporal dependency and non-stationarity of VCNs by adequately modeling the global temporal dependencies of the input samples and latent variables. Extensive experiments on the VCN anomaly detection dataset CIC-IDS2018 and the time-series anomaly detection benchmark dataset SMD show that GTGmVAE with MFM achieves the desirable performance, and GTGmVAE outperforms all nine representative state-of-the-art detection models.**

***Index Terms*—Virtual cloud network, anomaly detection, cloud security, network security, time-series data.**

I. Introduction

CLOUD computing [1] is a cornerstone of the modern digital economy, enabling applications to be deployed, managed and scaled with great flexibility and efficiency. As the fundamental resource of the cloud, the network in the cloud is a virtual network implemented through virtualization technologies for containers or VMs (commonly referred to as endpoints, EPs) to communicate with each other and with the outside of the cloud, which can be referred to as the virtual cloud network (VCN). VCN consists of two types of traffic: communication traffic between EPs, called east-west traffic, and traffic received and sent at the EP side to communicate with the outside of the cloud, called north-south traffic. VCN supports efficient and scalable data transfer and is the key link for businesses in the cloud to interact with each other and with the outside world. Therefore, the security of VCNs is critical. Unfortunately, VCNs have become the target of a variety of attacks [2], including attacks launched externally against VCNs, such as DDoS, brute force exploits, etc., and lateral movement attacks launched internally by compromised EPs to expand the compromised surface, such as port scanning, ransomware viruses, etc. These attacks cause serious consequences, such as leaking critical data and disrupting cloud services, resulting in significant losses. Therefore, intrusion detection for VCNs has become a key approach to securing the cloud.

Currently, related work implements intrusion detection for VCNs using signature based [3], [4] and supervised classification based [2], [5] methods. Unfortunately, these methods are difficult to actually apply to VCNs: signature based methods need to rely on a large number of known anomalous samples to form "signatures" and use these signatures to match samples to detect attacks, while supervised methods need to learn both normal and anomalous samples in order to find the decision boundary between them. However, it is extremely difficult to collect a sufficient number of representative anomalous samples with rich categories due to the diversity of attacks faced by VCNs. Meanwhile, signature based methods and supervised methods are usually unable to identify attack categories that are not in the training set. More seriously, VCNs are also exposed to unknown novel attacks, which cannot be identified by the above methods because they rely on existing anomalous samples. In contrast, unsupervised methods only learn the behavior pattern of normal samples, and any samples that deviate from the normal behavior pattern are considered as anomalous. Therefore, unsupervised methods fundamentally avoid the above problems. Theoretically, it is capable of detecting any category of attack, including novel attacks. In this paper, we focus on the unsupervised detection method for VCNs and try to learn only normal patterns and monitor for deviations from the normal patterns to detect attacks. This is known as anomaly based intrusion detection (hereafter referred to as VCN anomaly detection). Although several methods have been proposed for VCN anomaly detection, based on the observation and analysis of the characteristics of VCNs and the summary of the related work, we argue that VCN anomaly detection still faces the following key challenges:

Challenge 1 (C1): Design the scalable and temporally continuous feature model with the key consideration of the uniqueness of VCNs to effectively characterize

Received 24 May 2024; revised 21 February 2025; accepted 4 April 2025. Date of publication 16 April 2025; date of current version 29 April 2025. This work was supported by the National Key Research and Development Program of China under Grant 2023YFB3106303. The associate editor coordinating the review of this article and approving it for publication was Dr. Ning Zhang. *(Corresponding author: Chen Li.)*

The authors are with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100085, China, and also with the School of Cyber Security, University of Chinese Academy of Sciences, Beijing 100049, China (e-mail: mazixuan@iie.ac.cn; lichen@iie.ac.cn; zhangkun@iie.ac.cn; tubibo@iie.ac.cn).

This article has supplementary downloadable material available at https://doi.org/10.1109/TIFS.2025.3561672, provided by the authors.

Digital Object Identifier 10.1109/TIFS.2025.3561672

the network patterns of VCNs. VCNs are dominated by east-west traffic [6], which is significantly different from non-cloud networks dominated by north-south traffic, and poses the challenge of effectively characterizing the uniqueness of the network patterns of VCNs. However, current network traffic feature models [7], [8], [9], [10] mainly focus on non-cloud networks and are difficult to apply to VCNs. Meanwhile, current mainstream flow-based feature models [8], [9], [10] face the scalability challenge when used in VCNs typically containing large numbers of EPs: they need to maintain all the stateful flows for each EP to compute flow-level features. However, this not only leads to rapid growth of the state table with large computational and memory overheads [11], but also requires the traffic to be redirected to a dedicated middlebox for stateful flow processing, which suffers from low scalability, single point of failure, increased latency, and reduced throughput, and is infeasible in real clouds [2]. In addition, flow-based feature models focus on internal features of a single flow and construct temporally uncorrelated samples, and thus cannot be used by anomaly detection models to model the complex temporal dependency of VCNs (see **Challenge 3**). Therefore, it is necessary to fully consider the uniqueness of VCNs to effectively characterize their network patterns, to design stateless feature models for VCNs to cope with the scalability problem, and to construct temporally continuous feature samples for temporal modeling.

Challenge 2 (C2): Design the anomaly detection model with sufficient adaptability to learn the multiple heterogeneous normal patterns of VCNs simultaneously to address the key scalability problem. EPs oriented to different services have different network patterns. For example, the periodic peak traffic of Web EP vs. the uniform traffic of Database EP; and the uniform high traffic of Streaming Media Service EP vs. the random burst traffic of File Storage Service EP. Due to the heterogeneity of the network patterns of different EPs, current network anomaly detection methods have to follow the "one-to-one" strategy by constructing and maintaining an exclusive model for each EP that learns its normal network patterns. However, given the typically large number of EPs in VCNs and the high computational resources required by the detection models (typically deployed on dedicated hardware, e.g., GPU), this approach would incur huge training, inference, and model maintenance overheads, and thus has serious scalability issues in VCNs. For example, for a VCN containing 1,000 EPs, 1,000 models need to be trained, maintained and loaded separately and simultaneously on GPUs for online inference. In addition, EPs generated in batch, e.g. by dynamic scaling, are identical in business patterns and therefore have similar network patterns, and it is wasteful to train separate exclusive models for them. Therefore, there is a need to apply a new "one-to-many" [12], [13] strategy for VCN anomaly detection, where models are trained on samples from multiple EPs to simultaneously learn their normal network patterns. Only one trained model needs to be loaded for online detection, and samples from multiple EPs can be detected simultaneously. This effectively solves the key scalability problem. However, applying the "one-to-many" strategy poses a key challenge to the model's adaptability: current anomaly detection models [7], [14], [15], [16], [17], [18], [19] lack sufficient adaptability to simultaneously identify and learn multiple normal network patterns of different EPs, thus failing to achieve the desirable detection performance.

Challenge 3 (C3): Design the anomaly detection model with sufficient temporal modeling capability to cope with the complex temporal dependency and temporal non-stationarity of VCNs. Network traffic is typically time-series data, and the network patterns of VCNs have the complex temporal dependency, typically including: 1) trending, e.g. the steady increase and decrease of the business traffic carried by EPs; 2) periodicity: the stable periodic traffic patterns due to fixed business logics, e.g. the backend EP receives the frontend request and then triggers the invocation of the database. The complex temporal dependency of VCNs poses a challenge to the anomaly detection model's temporal modeling capability, and insufficient such capability or even ignoring the temporal dependency will make it difficult to characterize the network patterns of VCNs. However, current related work either ignores the temporal dependency [7], [20] or has insufficient capability to model it [12], [13], [14], [15], [16], [17], [18], [19], [21], [22], thus failing to achieve the desirable detection performance. On top of that, VCNs also exhibit temporal non-stationarity, i.e., the network patterns of a single EP have multiple temporal dependencies that change dynamically over time. Typical examples include: 1) EPs hosting cloud services have different network patterns during day and night due to different user activities; 2) dynamic changes in network patterns due to business adjustments and upgrades of EPs; and 3) EPs (e.g. VMs) running different applications simultaneously have different network patterns at different times. This poses the further challenge of requiring not only that the model has the adaptability in **C2**, but also that it is able to accurately capture the dynamic changes in network patterns while effectively modeling the complex temporal dependency. However, current related work either lacks the adaptability [7], [14], [15], [16], [17], [18], [19] or fails to accurately capture the dynamic changes in network patterns and effectively model the complex temporal dependency [12], [13], [20], [21], [22], thus failing to achieve the desirable detection performance.

To address the above challenges, we model VCN anomaly detection as an unsupervised time-series anomaly detection problem and apply the "one-to-many" strategy. Based on this, we propose a new unsupervised time-series VCN anomaly detection method that consists of two parts:

(1) A new **M**ultilevel **F**eature **M**odel (MFM) for VCN anomaly detection. MFM includes three levels of features, besides the basic packet frequency and throughput features, new topology features are proposed based on the observation and analysis of the uniqueness of VCNs. By combining the multilevel features, MFM provides a comprehensive and accurate characterization of the network patterns of VCNs. Meanwhile, MFM requires only simple packet header statistics to achieve stateless EP-level feature construction and constructs temporally continuous samples for temporal modeling using the time window. Therefore, MFM can effectively address **C1**.

(2) A new **G**lobal **T**emporal Dependency **G**aussian **m**ixture Dynamic **V**ariational **A**uto-**E**ncoder (GTGmVAE) model for unsupervised time-series VCN anomaly detection,

which contains several innovations: a) GTGmVAE contains a new local-global feature extractor that can effectively extract and integrate the implicit structural features of the input samples from both local and global perspectives. This can effectively improve the model's representation capability and make the model better learn the distribution of the input samples, thus improving the model's adaptability and temporal modeling capability. b) GTGmVAE contains a new global temporal dependency Gaussian mixture inference network, and a new global latent-space temporal dependency Gaussian mixture generation network. GTGmVAE achieves strong temporal inference capability by capturing the global temporal dependency of the input samples, and strong temporal sample generation capability by capturing the global temporal dependency of the latent variables. This can effectively improve the capability to model the complex temporal dependency of VCNs, and help improve the model's adaptability and the capability to cope with the temporal non-stationarity of VCNs. Meanwhile, GTGmVAE makes the latent variable follow a Gaussian mixture distribution, which is more adaptive than a single distribution: different network patterns can be corresponded to different categories of the latent variable distributions, and the categories of the distributions to which the latent variables belong are inferred based on the global temporal dependency of the input samples to accurately capture the dynamic changes of the network patterns. Therefore, GTGmVAE is able to model both the multiple network patterns of different EPs as well as the multiple network patterns of a single EP. Taken together, GTGmVAE can effectively address **C2** and **C3**.

Our contributions are as follows:

(1) We propose a new multilevel feature model MFM (§ III) for VCN anomaly detection, which requires only simple packet header statistics for the stateless EP-level feature construction, and contains three levels of features: packet frequency features, throughput features, and new topology features designed for the uniqueness of VCNs. Combined with the multilevel features, MFM provides a comprehensive and accurate characterization of the network patterns of VCNs.

(2) We propose a new unsupervised time-series anomaly detection model GTGmVAE (§ IV) for VCN anomaly detection: a) GTGmVAE contains a new feature extractor that can adequately extract and integrate the implicit structural features of the input samples from both global and local perspectives to effectively improve the model's representation capability and better implement the temporal modeling. b) GTGmVAE has a strong temporal modeling capability. By fully capturing the global temporal dependency of the input samples and latent variables, GTGmVAE obtains the strong temporal inference capability and temporal generation capability. c) GTGmVAE has a strong adaptability. By making the latent variable follow a Gaussian mixture distribution and based on the global temporal dependency of the input samples to accurately infer the distribution categories of the latent variables, on top of which combining the model's strong temporal modeling capability with the representation capability provided by the new feature extractor, GTGmVAE is able to simultaneously learn the multiple network patterns of different EPs and effectively cope with the temporal non-stationarity of VCNs.

(3) We conduct extensive experiments (§ V) on the VCN anomaly detection dataset CIC-IDS2018 [23] and the time-series anomaly detection benchmark dataset SMD [15], which fully illustrate the effectiveness of GTGmVAE and MFM in VCN anomaly detection: a) GTGmVAE outperforms all nine state-of-the-art representative methods in the comparison experiments, illustrating its advantages in temporal modeling capability and adaptability; b) ablation experiments on GTGmVAE illustrate the effectiveness of each component in GTGmVAE; c) ablation experiments on MFM illustrate the effectiveness of each level of features in MFM; and d) time efficiency experiments illustrate that GTGmVAE and MFM can meet the real-time detection requirement.

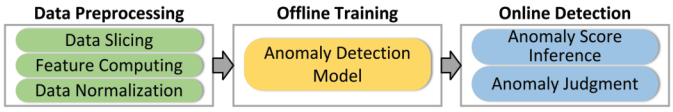

Fig. 1. VCN anomaly detection framework.

II. Preliminaries

A. Problem Definition

Since the "one-to-many" strategy [12], [13] is used, the sample sequence of n-th EP is defined as $x_n = \{x_{1,n}, x_{2,n}, \cdots, x_{T,n}\}$, where $n = 1, \cdots, N$, N is the number of EPs, and T is the sequence length of x_n. The observation sample at time t is $x_{t,n} \in \mathbb{R}^v$, where v is the dimension of the sample, i.e., the number of features of the feature model. Time-series anomaly detection for VCNs is defined as a task that determines whether a specific sample $x_{t,n}$ from a specific EP is anomalous or not. On top of that, the "one-to-many" strategy means that the model is trained on samples from all N EPs to obtain a unified model, and then it is used to determine whether any of the samples from any of the N EPs is anomalous or not. In contrast, the traditional "one-to-one" strategy involves training N exclusive models for N EPs, and n-th model can only be used to determine whether the sample from n-th EP is anomalous or not. Note that in the following description, for simplicity, we refer to $x_{t,n}$ simply as x_t.

B. Overview of the Framework

The anomaly detection framework (Fig. 1) is divided into three parts. 1) Data pre-processing. VCN traffic is natural time-series data, so we first slice it using the time window. Second, we compute features for the traffic in each time window using the proposed MFM (§ III) to construct temporally continuous training and test samples for each EP. Finally, we normalize the samples using MinMax to facilitate model training and inference. 2) Offline training. We feed the preprocessed training samples into the proposed GTGmVAE (§ IV) for training. 3) Online detection. We use the trained model to infer the anomaly score of the samples from each EP online and use the threshold to determine whether they are anomalies.

C. VAE and Dynamic Variants

Variational auto-encoder (VAE) [24] is a deep generative model consisting of an inference network and a generation network. Inference network aims to map the input x to a latent space and the distribution (usually Gaussian) of the latent variable z is parameterized as the mean μ and variance σ^2. This can be denoted as $q_\phi(z|x)$. Generation network aims to reconstruct the input x from z sampled from the latent space, denoted as $p_\theta(x|z)$. The ability of VAE to capture the intrinsic pattern of the input by learning its latent representation makes it particularly suitable for anomaly detection: anomalies often significantly differ from normal samples in patterns, and by comparing the reconstruction loss of the samples, VAE can effectively identify anomalies. The loss function of VAE is shown below and consists of two parts: the log-likelihood of the reconstructed x measures the reconstruction effect, while the KL divergence measures the difference between the encoded latent distribution and the prior distribution.

$$\mathcal{L}(\theta,\phi;x) = \mathbb{E}_{q_\phi(z|x)}\big[\log p_\theta(x|z)\big] - \mathrm{KL}\big(q_\phi(z|x)\,\|p_\theta(z)\big) \quad (1)$$

Dynamic variants of VAE have been extensively studied for processing time-series data. They usually involve modifications to the structure of VAE, such as introducing recurrent structures (e.g., RNN) into VAE for temporal modeling. The most typical work is VRNN [14], which uses RNN to temporally model the input sequence x and the latent variable sequence z, i.e., $h_t = \mathrm{RNN}(x_t, z_t, h_{t-1})$, and infers the posterior distribution $q_\phi(z_t|x_t, h_{t-1})$ of the latent variable z_t based on the temporal hidden variable h_{t-1}. Generation network samples z_t from $p_\theta(z_t|h_{t-1})$ and implements the reconstruction $p_\theta(x_t|z_t, h_{t-1})$ in combination with h_{t-1}. Therefore, VRNN is not only able to learn the latent representation of the input, but also to capture its evolution over time, thus enabling temporal modeling. The loss function of VRNN is similar to that of VAE at each time t. We can conclude from the design of VRNN that modifying the structure of VAE to enable temporal modeling requires implementing temporal modeling of the input sequence x and the latent variable sequence z, and making the model reconstruct the sequence x.

III. Multi-Level Feature Model (MFM)

This section details the proposed new multilevel feature model MFM specifically designed for VCN anomaly detection, which combines three levels of features. First, we use the feature construction approach of Aldribi et al. [18] to construct the packet frequency features as Level 1 features. Since Level 1 features only consider the packet count information, which is too homogeneous, we further propose the new throughput features as Level 2 features. Level 1 and 2 features characterize the basic patterns of general networks, which is similar to the current feature models [2], [7], but is not sufficient for VCN anomaly detection as described in **C1**. Therefore, by observing and analyzing the uniqueness of VCNs, we propose the new topology features as Level 3 features. Combined with the multilevel features, MFM can comprehensively and accurately characterize the network patterns of VCNs. Meanwhile, MFM constructs the temporally continuous feature samples for EPs using the time window to allow anomaly detection models to model the complex temporal dependency of VCNs. In addition, MFM requires only simple packet header statistics to enable the stateless feature construction, thus avoiding the use of dedicated middleboxes for stateful flow processing. In summary, MFM can effectively address **C1**.

TABLE I
Basic Frequency Features in the First Level

Ingress Direction	Egress Direction
$f^{in}_{sip,sp,eip,dp}(t) = PC_{in}(t,\Delta t)/\Delta t$	$f^{out}_{eip,sp,dip,dp}(t) = PC_{out}(t,\Delta t)/\Delta t$
$f^{in}_{sp,eip,dp}(t) = \bigcup_{\forall sip} PC_{in}(t,\Delta t)/\Delta t$	$f^{out}_{eip,sp,dp}(t) = \bigcup_{\forall dip} PC_{out}(t,\Delta t)/\Delta t$
$f^{in}_{sip,eip,dp}(t) = \bigcup_{\forall sp} PC_{in}(t,\Delta t)/\Delta t$	$f^{out}_{eip,sp,dip}(t) = \bigcup_{\forall dp} PC_{out}(t,\Delta t)/\Delta t$
$f^{in}_{sip,sp,eip}(t) = \bigcup_{\forall dp} PC_{in}(t,\Delta t)/\Delta t$	$f^{out}_{eip,dip,dp}(t) = \bigcup_{\forall sp} PC_{out}(t,\Delta t)/\Delta t$
$f^{in}_{sip,eip}(t) = \bigcup_{\forall sp,dp} PC_{in}(t,\Delta t)/\Delta t$	$f^{out}_{eip,dip}(t) = \bigcup_{\forall sp,dp} PC_{out}(t,\Delta t)/\Delta t$
$f^{in}_{sp,eip}(t) = \bigcup_{\forall sip,dp} PC_{in}(t,\Delta t)/\Delta t$	$f^{out}_{eip,dp}(t) = \bigcup_{\forall sp,dip} PC_{out}(t,\Delta t)/\Delta t$
$f^{in}_{eip,dp}(t) = \bigcup_{\forall sip,sp} PC_{in}(t,\Delta t)/\Delta t$	$f^{out}_{eip,sp}(t) = \bigcup_{\forall dip,dp} PC_{out}(t,\Delta t)/\Delta t$
$f^{in}_{eip}(t) = \bigcup_{\forall sip,sp,dp} PC_{in}(t,\Delta t)/\Delta t$	$f^{out}_{eip}(t) = \bigcup_{\forall sp,dip,dp} PC_{out}(t,\Delta t)/\Delta t$

A. Level 1: Packet Frequency Features

We use the approach of Aldribi et al. [18] to construct the packet frequency features, including basic frequency, load and entropy features. Specifically, assume that the IP address of the EP for which features are constructed is *eip*. The network flow of *eip* can be divided into two categories based on its directions. The egress flow is represented by the tuple (eip, sp, dip, dp) (protocol is ignored in the representation for convenience, same below), where *sp* is the source port, *dip* is the destination IP, and *dp* is the destination port. The ingress flow is represented by the tuple (sip, sp, eip, dp), where *sip* is the source IP, *sp* is the source port, and *dp* is the destination port. Note that the definition of such a flow requires only simple packet header statistics and avoids maintaining flow state, i.e., it is stateless. Let $PC_{out}(t,\Delta t)$ and $PC_{in}(t,\Delta t)$ represent the number of packets in the egress/ingress flow at time $(t, t+\Delta t)$, and the size of the time window is Δt, then the basic frequency features are defined as shown in table I.

Based on basic frequency features, frequency load features calculate the ratio between the corresponding ingress and egress frequency features: $LF_{j,j'}(t) = f^{out}_{j'}(t)/f^{in}_{j}(t)$. Each row in table I represents the corresponding $f^{in}_{j}(t)$ and $f^{out}_{j'}(t)$.

Based on basic frequency features, frequency entropy features calculate the randomness of the packet frequency, which is defined as: $HF_f(t) = -\sum_i (f_i/f)\log_2(f_i/f)$, where i represents the variable in basic frequency features. For example, consider $f^{in}_{sip,eip}(t)$, where *eip* is fixed since we construct features for the EP corresponding to *eip*, but *sip* is not fixed and can have multiple values. Therefore we have $HF(f^{in}_{sip,eip}, t) = -\sum_{sip}\left(f^{in}_{sip,eip}/\sum_{sip}(f^{in}_{sip,eip})\right)\log_2\left(f^{in}_{sip,eip}/\sum_{sip}(f^{in}_{sip,eip})\right)$.

The above three features characterize the normal patterns of the packet frequency and are used as Level 1 features in MFM. Among them, basic frequency features assume that the packet frequency should be stable, load features assume that the ratio between ingress and egress packet frequency should be stable, and entropy features assume that the packet frequency should have low randomness to distinguish from attacks with high randomness (e.g. DDoS). However, Level 1 features are not sufficient as they only consider the packet count information.

TABLE II
BASIC THROUGHPUT FEATURES IN THE SECOND LEVEL

Ingress Direction	Egress Direction
$tp^{in}_{sip,sp,eip,dp}(t) = FL_{in}(t,\Delta t)/\Delta t$	$tp^{out}_{eip,sp,dip,dp}(t) = FL_{out}(t,\Delta t)/\Delta t$
$tp^{in}_{sp,eip,dp}(t) = \bigcup_{\forall sip} FL_{in}(t,\Delta t)/\Delta t$	$tp^{out}_{eip,sp,dp}(t) = \bigcup_{\forall dip} FL_{out}(t,\Delta t)/\Delta t$
$tp^{in}_{sip,eip,dp}(t) = \bigcup_{\forall sp} FL_{in}(t,\Delta t)/\Delta t$	$tp^{out}_{eip,sp,dip}(t) = \bigcup_{\forall dp} FL_{out}(t,\Delta t)/\Delta t$
$tp^{in}_{sip,sp,eip}(t) = \bigcup_{\forall dp} FL_{in}(t,\Delta t)/\Delta t$	$tp^{out}_{eip,dip,dp}(t) = \bigcup_{\forall sp} FL_{out}(t,\Delta t)/\Delta t$
$tp^{in}_{sip,eip}(t) = \bigcup_{\forall sp,dp} FL_{in}(t,\Delta t)/\Delta t$	$tp^{out}_{eip,dip}(t) = \bigcup_{\forall sp,dp} FL_{out}(t,\Delta t)/\Delta t$
$tp^{in}_{sp,eip}(t) = \bigcup_{\forall sip,dp} FL_{in}(t,\Delta t)/\Delta t$	$tp^{out}_{eip,dp}(t) = \bigcup_{\forall sp,dip} FL_{out}(t,\Delta t)/\Delta t$
$tp^{in}_{eip,dp}(t) = \bigcup_{\forall sip,sp} FL_{in}(t,\Delta t)/\Delta t$	$tp^{out}_{eip,sp}(t) = \bigcup_{\forall dip,dp} FL_{out}(t,\Delta t)/\Delta t$
$tp^{in}_{eip}(t) = \bigcup_{\forall sip,sp,dp} FL_{in}(t,\Delta t)/\Delta t$	$tp^{out}_{eip}(t) = \bigcup_{\forall sp,dip,dp} FL_{out}(t,\Delta t)/\Delta t$

B. Level 2: Throughput Features

To further comprehensively and accurately characterize the basic patterns of VCNs, we extend Level 1 features and propose the throughput features as Level 2 features. Similarly, it consists of basic throughput, throughput load and throughput entropy features. Let $FL_{out}(t,\Delta t)$ and $FL_{in}(t,\Delta t)$ represent the flow length (i.e., sum of packet size) of the egress/ingress flow at time $(t, t+\Delta t)$, and the size of the time window is Δt, the basic throughput features are defined as shown in table II.

Based on basic throughput features, throughput load features calculate the ratio between the corresponding ingress and egress throughput features: $LT_{j,j'}(t) = tp^{out}_{j'}(t)/tp^{in}_{j}(t)$. Each row in table II represents corresponding $tp^{in}_{j}(t)$ and $tp^{out}_{j'}(t)$.

Based on basic throughput features, throughput entropy features calculate the randomness of the flow throughput, which is defined as: $HT_{tp}(t) = -\sum_{i}\left(tp_i/tp\right)\log_2\left(tp_i/tp\right)$, where i represents the variable in basic throughput features.

Throughput features assume that the throughput and its load and entropy of different levels of flows should be stable. Compared to Level 1 features that only consider packet count information, it can effectively distinguish anomalies in packet size, e.g. the packet from brute force attacks has a different size from the packet from normal business traffic. By combining Level 1 and 2 features that consider the flow length and packet count of different levels of flows, we can comprehensively and accurately characterize the basic patterns of VCNs.

C. Level 3: Topology Features

Level 1 and 2 features ignore the uniqueness of VCNs, so they are still insufficient for VCN anomaly detection as described in **C1**. Therefore, by observing, analyzing and summarizing the network patterns of VCNs, we identify the uniqueness of VCNs: VCNs are dominated by east-west traffic typically generated by accesses between services hosted by EPs [2], [18], [23], [25], such as invocations between the containerized microservices and accesses between the frontend and backend hosted by VMs. Since the business logic is fixed, the connection behavior of an EP, i.e., the edges contained by that EP in the connection topology graph, should have a stable baseline pattern. In contrast, the connection behavior of an EP will deviate from the baseline when attacks occur, including: when this EP is exposed to external DDoS attacks or internal lateral movement attacks such as port scanning from compromised EPs, or when this EP is compromised and launches lateral movement attacks against other EPs.

We further illustrate this with two examples in microservices (container cloud) and VM cloud respectively. Fig. 2 shows the

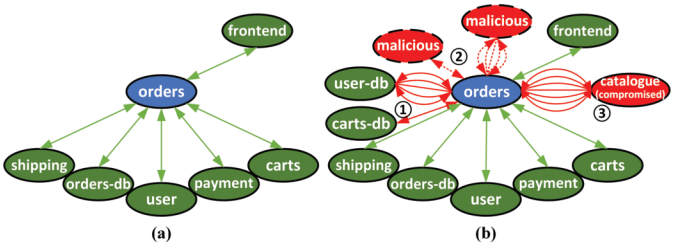

Fig. 2. Comparison of the normal and anomalous behaviors of the microservice application Sock Shop [26]. (a) is the connection topology graph of the microservice *orders* in a normal situation, and (b) is the connection topology graph with three examples of attacks. Red nodes are internal EPs launching the attacks and external attackers. Red lines represent attacks.

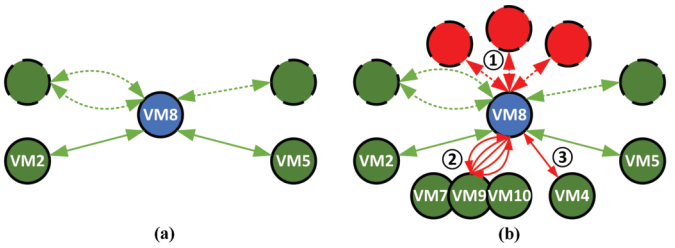

Fig. 3. Comparison of the normal and anomalous behaviors of VMs in the cloud [18]. (a) is the connection topology graph of VM8 in a normal situation, and (b) is the connection topology graph with three real-world attacks. Red nodes are external attackers. Red lines represent attacks.

connection topology graph of the containerized microservice *orders* in the classic microservice application Sock Shop [26]. Fig. 2(a) is the connection topology graph of *orders* in a normal situation, which has a stable pattern as the business logic is fixed. In contrast, Fig. 2(b) shows the connection topology graph containing three assumed attack examples, where ① is the attack launched by compromised *orders* to other EPs within Sock Shop (with which *orders* should not communicate in the business logic), ② is the attack from outside the cloud to the inside, and ③ is the attack launched by other compromised EPs in the cloud (e.g. the microservice *catalogue*) to *orders*. Fig. 3 shows the real connection topology graph of VM8 on the first day from ISOT-CID dataset [18]. VM8 has a stable topology graph pattern in a normal situation (Fig. 3(a)). In contrast, the topology graph changes significantly when real-world attacks occur (Fig. 3(b)), including ① attacks launched by an external attacker on VM8, leading to the compromise of VM8; ② a port scanning attack launched by VM8 on VMs 7, 9 and 10, and ③ a DoS attack launched by VM8 on VM4.

Based on the above observations, we arrive at the key insight: characterizing the relatively stable normal patterns of the EP's connection topology graph can effectively discriminate between normal and anomalous behaviors. Therefore, we propose the topology features as Level 3 features, which characterize the EP's connection topology graph from different perspectives in as simple a way as possible to avoid additional computational overhead. Specifically, the network flows that *eip* receives and sends out during a time window of time t and length Δt can be denoted as $F_{out}(t,\Delta t)$ and $F_{in}(t,\Delta t)$. We further determine whether the traffic belongs to inside or outside the cloud, i.e., whether it is east-west (*ew*) traffic or north-south (*ns*) traffic, based on the *dip* and *sip*. Finally, we obtain four types of flows: $F^{ew}_{out}(t,\Delta t)$, $F^{ns}_{out}(t,\Delta t)$, $F^{ew}_{in}(t,\Delta t)$

TABLE III
TOPOLOGY FEATURES

Features	Meaning
$tf_{all}(t) = \lvert\cup_{\forall sp,dip,dp} F_{out}(t,\Delta t)\rvert + \lvert\cup_{\forall sip,sp,dp} F_{in}(t,\Delta t)\rvert$	Number of flows in Δt
$tf_{ew}(t) = \lvert\cup_{\forall sp,dip,dp} F^{ew}_{out}(t,\Delta t)\rvert + \lvert\cup_{\forall sip,sp,dp} F^{ew}_{in}(t,\Delta t)\rvert$	Number of ew flows in Δt
$tf_{ns}(t) = \lvert\cup_{\forall sp,dip,dp} F^{ns}_{out}(t,\Delta t)\rvert + \lvert\cup_{\forall sip,sp,dp} F^{ns}_{in}(t,\Delta t)\rvert$	Number of ns flows in Δt
$tf_{in}(t) = \lvert\cup_{\forall sip,sp,dp} F^{ew}_{in}(t,\Delta t)\rvert + \lvert\cup_{\forall sip,sp,dp} F^{ns}_{in}(t,\Delta t)\rvert$	Number of ingress flows in Δt
$tf_{out}(t) = \lvert\cup_{\forall sp,dip,dp} F^{ew}_{out}(t,\Delta t)\rvert + \lvert\cup_{\forall sp,dip,dp} F^{ns}_{out}(t,\Delta t)\rvert$	Number of egress flows in Δt
$tf^{ew}_{in}(t) = \lvert\cup_{\forall sip,sp,dp} F^{ew}_{in}(t,\Delta t)\rvert$	Number of ingress ew flows in Δt
$tf^{ew}_{out}(t) = \lvert\cup_{\forall sp,dip,dp} F^{ew}_{out}(t,\Delta t)\rvert$	Number of egress ew flows in Δt
$tf^{ns}_{in}(t) = \lvert\cup_{\forall sip,sp,dp} F^{ns}_{in}(t,\Delta t)\rvert$	Number of ingress ns flows in Δt
$tf^{ns}_{out}(t) = \lvert\cup_{\forall sp,dip,dp} F^{ns}_{out}(t,\Delta t)\rvert$	Number of egress ns flows in Δt
$tf_{L1}(t) = tf_{ew}(t)/tf_{ns}(t)$	Ratio of $tf_{ew}(t)$ to $tf_{ns}(t)$
$tf_{L2}(t) = tf_{in}(t)/tf_{out}(t)$	Ratio of $tf_{in}(t)$ to $tf_{out}(t)$
$tf_{L3}(t) = tf^{ew}_{in}(t)/tf^{ew}_{out}(t)$	Ratio of $tf^{ew}_{in}(t)$ to $tf^{ew}_{out}(t)$
$tf_{L4}(t) = tf^{ns}_{in}(t)/tf^{ns}_{out}(t)$	Ratio of $tf^{ns}_{in}(t)$ to $tf^{ns}_{out}(t)$
$tf_{L5}(t) = tf^{ew}_{in}(t)/tf^{ns}_{in}(t)$	Ratio of $tf^{ew}_{in}(t)$ to $tf^{ns}_{in}(t)$
$tf_{L6}(t) = tf^{ew}_{out}(t)/tf^{ns}_{out}(t)$	Ratio of $tf^{ew}_{out}(t)$ to $tf^{ns}_{out}(t)$

and $F^{ns}_{in}(t,\Delta t)$. On top of that, topology features simply and effectively characterize the EP's connection topology graph by counting the number of different types of flows and the ratios between them, which are defined as shown in table III. In addition, in some special cases, distributed applications may be deployed in non-VCNs, such as deploying internal business systems in a private data center or deploying small applications in a LAN environment. These special networks may exhibit uniqueness similar to that of VCNs. However, such cases are usually rare. Meanwhile, MFM is theoretically applicable.

D. Feature Computation

Having defined MFM, we now define how the features are computed. For a time period T, we slice it by continuous and disjoint time windows and compute all the features in each time window $(t, t+\Delta t)$, totalling 91 dimensions. Finally, we obtain a series of temporally continuous samples for training and online detection. From the specific definition of MFM, we can see that MFM only requires statistics on flow tuples (i.e., (eip, sp, dip, dp) or (sip, sp, eip, dp)) and packet sizes in the packet headers, and therefore no stateful flow processing is required. We further discuss the flexibility and challenges of deploying MFM in the cloud in § VI-A.

IV. VCN ANOMALY DETECTION MODEL GTGMVAE

This section details the proposed new VCN anomaly detection model GTGmVAE. It consists of a new global temporal dependency Gaussian mixture inference network, which contains a new feature extractor, and a new global latent-space temporal dependency Gaussian mixture generation network.

A. Global Temporal Dependency Gaussian Mixture Inference Network

This section details the proposed new global temporal dependency Gaussian mixture inference network, which is

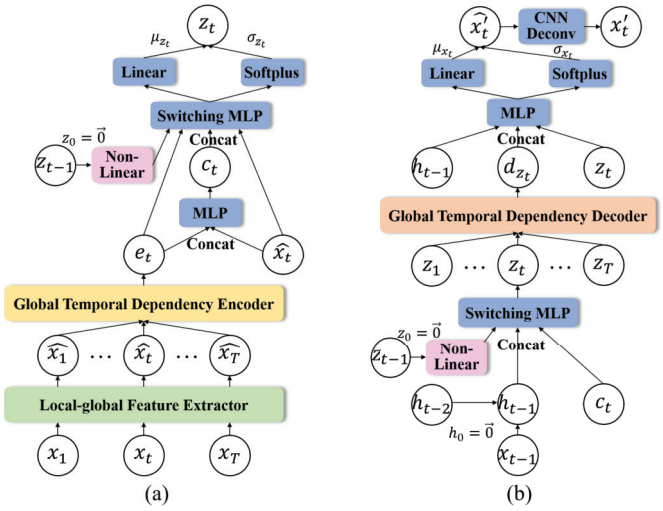

Fig. 4. Network structure of GTGmVAE. (a) Inference network. (b) Generation network.

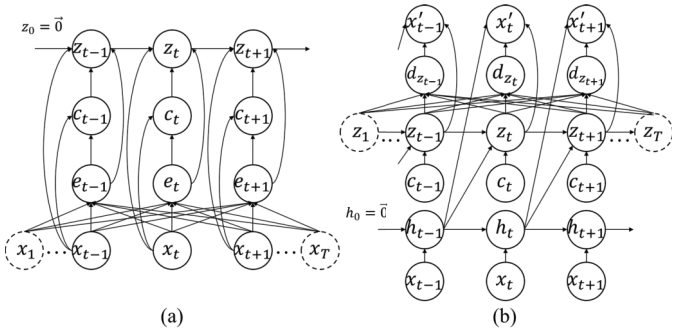

Fig. 5. Probabilistic graphical model of GTGmVAE. (a) Inference network. (b) Generation network.

used to infer the latent variable sequence z. Combined with its probabilistic graphical model (Fig. 5(a)), we give the joint probability distribution of the inference network:

$$\begin{aligned} &q_\phi(z_{1:T}, e_{1:T}, c_{1:T} | x_{1:T}, z_0) \\ &= q_\phi(z_{1:T} | e_{1:T}, c_{1:T}, x_{1:T}, z_0) q_\phi(c_{1:T} | x_{1:T}, e_{1:T}) q_\phi(e_{1:T} | x_{1:T}) \\ &= \prod_{t=1}^{T} q_\phi(z_t | z_{t-1}, e_t, c_t, x_t)\, q_\phi(c_t | x_t, e_t) q_\phi(e_t | x_{1:T}) \end{aligned} \tag{2}$$

In the following, we describe the computational process based on the structure of the inference network (Fig. 4(a)).

1) Local-Global Feature Extractor: In VAE [24] and dynamic VAE (e.g., VRNN [14]), the input sample x is processed via multiple fully connections (also known as the multilayer perceptron, MLP) to: 1) extract features from input samples to obtain high-level representations while eliminating feature redundancy; and 2) compress dimension to facilitate subsequent latent variable inference and temporal modeling. However, given the high-dimensional nature of the samples, this approach ignores the implicit structural features of input samples, thus affecting subsequent operations; and makes the model lack the sufficient representation capability, preventing it from achieving desirable adaptability. Some advanced work [12], [27] uses the one-dimensional convolutional neural network (1D CNN) to extract the implicit structural features. However, due to the limitations of convolutional operations,

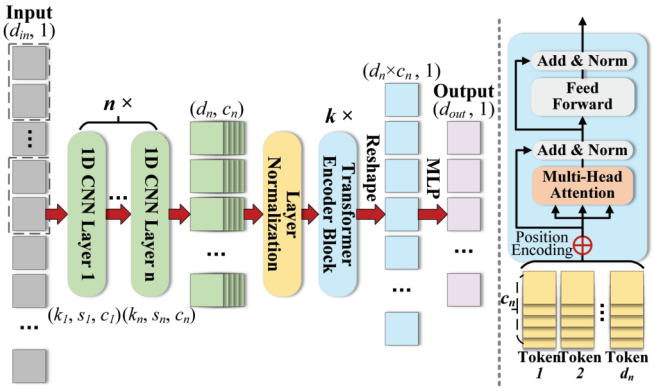

Fig. 6. Network structure of the Local-global Feature Extractor (LGFE).

they can only process local features and ignore the global perspective.

Therefore, we propose a new Local-global Feature Extractor (LGFE) (Fig. 6). Specifically, each sample $x_t \in \mathbb{R}^v$ in the input sequence x is a v-dimensional vector. We use each x_t as an input vector to LGFE, i.e., $d_{in} = v$. First, x_t is processed by n layers of 1D CNN, and the parameters of each layer are (k_i, s_i, c_i), representing its kernel size, step size, and output channels, where $s_i > 1$, and c_i gradually increases. This can: 1) effectively extract and integrate local features of x_t, and 2) reduce the length of the *token* sequence (from d_{in} to d_n) while increasing the dimension of each *token* (from 1 to c_n), which can effectively reduce the computational overhead of the subsequent global feature processing while representing *token* over a larger feature space to increase the richness of feature representation. Then, we perform the layer normalization on the output of n-th layer 1D CNN and obtain a *token* sequence of length d_n and dimension c_n. To process global features, we use Transformer Encoder [28], which effectively extracts global features and enhances representation capability through a self-attention mechanism. Similarly, we obtain the processed *token* sequence by stacking k Transformer Encoder Blocks, whose dimension is still (d_n, c_n). We perform the reshape operation on the *token* sequence to transform it into a vector of $(d_n \times c_n, 1)$ and obtain the output vector $\widehat{x}_t$ with dimension $(d_{out}, 1)$ by an MLP $((d_n \times c_n) \rightarrow (d_{inner}) \rightarrow (d_{out}))$, where $d_{inner} \geq d_n \times c_n$, $d_{out} < d_{in}$. Here MLP has the effect of further integrating global and local features and performing dimension compression. Finally, we combine local and global perspectives to achieve feature extraction and integration of x_t and obtain its compressed representation $\widehat{x}_t$, which is able to (a) fully extract and integrate the features of input samples and eliminate feature redundancy; (b) contribute to better subsequent latent variable inference and temporal dependency modeling; and (c) effectively improve the representation capability of GTGmVAE to achieve the desirable adaptability.

2) Global Temporal Dependency Encoder: Having obtained the compressed representation $\widehat{x} = \{\widehat{x}_1, \widehat{x}_2, \cdots, \widehat{x}_T\}$, GTGmVAE further models its temporal dependency. Current dynamic VAEs such as VRNN [14] and SGmVRNN [12] usually implement temporal modeling via RNN, but they have two problems: (a) RNN only consider historical information and cannot obtain the global temporal dependency; (b) using the same RNN to model temporal dependencies of both deterministic input samples and stochastic latent variables, and their interactions will degrade the modeling performance. In oder to obtain a strong temporal modeling capability to effectively address **C3**, we aim to obtain the global temporal dependency of sequence $\widehat{x}$, which can fully exploit the information about the current time sample $\widehat{x}_t$ itself, the past and the future to better infer z_t. Moreover, the temporal modeling of the deterministic state $\widehat{x}_t$ and the stochastic state z_t is separated (§ IV-A.4). Specifically, from the knowledge of the probabilistic graphical model (Fig. 5(a)), it is known that the temporal hidden variable sequence $e = \{e_1, e_2, \cdots, e_T\}$ is calculated from all $\widehat{x}_t$ (i.e., $q_\phi(e_t|x_{1:T})$), and is thus able to model the global temporal dependency, which is achieved by the new Global Temporal Dependency Encoder (GTDE) in Fig. 4(a). GTDE is stacked by several Transformer Encoder Blocks [28], and through the self-attention mechanism, it can effectively capture the global temporal dependencies of samples in $\widehat{x}_t$ with each other, and process long sequences better than RNN, thus effectively modeling the complex temporal dependency of VCNs. The computational procedure of GTDE is as follows:

$$x^i_{input} = \begin{cases} \widehat{x}, & i = 1 \\ e^{i-1}, & i > 1 \end{cases} \tag{3}$$

$$a^i = \text{MultiHeadAttntion}\left(x^i_{input}, x^i_{input}, x^i_{input}\right) \tag{4}$$

$$a^i_{norm} = \text{LayerNorm}\left(a^i + x^i_{input}\right) \tag{5}$$

$$e^i = \text{LayerNorm}\left(\text{FeedFoward}\left(a^i_{norm}\right) + a^i_{norm}\right) \tag{6}$$

By cyclically performing the above computational procedure N times, we take the final e^N as the global temporal hidden variable sequence $e = \{e_1, e_2, \cdots, e_T\}$.

3) Inference of Indicator Variable c: To learn the multiple normal network patterns of different EPs as required by the "one-to-many" strategy in **C2**, and to cope with the temporal non-stationarity in **C3**, we need to accurately perceive different normal network patterns. Inspired by GmVAE [20] and SGmVRNN [12], we denote the categories of the latent variable distributions corresponding to different normal patterns by introducing an indicator variable c, and capture the different normal patterns by inferring the categories of c. Specifically, c is a one-hot vector and follows a Categorical distribution with parameter $\pi \in \mathbb{R}_+^{K\times 1}$, i.e., $c \sim \text{Cat}(\pi)$. Since changes in the normal network patterns of EPs are temporally correlated, to accurately infer the indicator variable c_t, we need to consider not only the current time sample $\widehat{x}_t$, but also the global temporal dependency between $\widehat{x}_t$ and other samples in $\widehat{x}$. However, existing work either ignores temporal dependency [20] or considers only the historical information while ignoring the future information [12]. Therefore, to fully consider the global temporal dependency of $\widehat{x}_t$ to accurately infer c_t, we use the global temporal hidden variable sequence e obtained in § IV-A.2, where e_t represents the global dependency between $\widehat{x}_t$ and other samples in $\widehat{x}$. Specifically, c_t is calculated by:

$$\pi_t = \text{Softmax}\left(\text{MLP}\left(\text{Concat}\left(\widehat{x}_t,\ e_t\right)\right)\right) \tag{7}$$

$$c_t = \text{GumbleSoftmax}\left(\pi_t\right) \tag{8}$$

where we need to obtain the discrete distribution variable c_t. However, since it is unable to backpropagate the gradient directly from the discrete distribution, the Gumble-Softmax trick [29] is introduced here to approximate the discrete

distribution, whose differentiability guarantees that parameter optimization can be performed by backpropagation.

4) Inference of Latent Variable z: Since we introduce the indicator variable c to indicate the distribution category of the latent variable z, the prior distribution of z is extended to a Gaussian mixture distribution (§ IV-B.1). Accordingly, the posterior distribution of z also depends on c and is extended to a Gaussian mixture distribution. Combining the knowledge of the probabilistic graphical model (Fig. 5(a)) and Fig. 4(a), it is known that the variational posterior of the current time latent variable z_t is inferred from four parts: (a) the previous time latent variable z_{t-1}, which introduces its own temporal dependency for the variational posterior inference of z. Meanwhile, the temporal dependency of the stochastic sequence z is modelled separately from that of the deterministic sequence $\widehat{x}$ to avoid mutual influence. Specifically, for z_{t-1}, we obtain its nonlinear temporal dependency factor by $\widehat{z_{t-1}} = \text{Tanh}\left(\text{Linear}\left(z_{t-1}\right)\right)$ and use $\widehat{z_{t-1}}$ to infer the posterior of z_t. (b) The current time input $\widehat{x}_t$, which introduces information of the current time sample for variational inference. (c) The current time global temporal hidden variable e_t, which represents the global temporal dependency of the input sequence $\widehat{x}$ to accurately infer the posterior of z_t. (d) The current time indicator variable c_t, which indicates the category of the posterior distribution of z_t, thus providing GTGmVAE with the adaptability to model different normal patterns. Taken together, the variational posterior of z_t is calculated by:

$$\mu_{\phi,z_t} = \text{MLP}\left(\text{Concat}\left(\widehat{z_{t-1}}, \widehat{x}_t,\ e_t,\ c_t\right)\right) \tag{9}$$

$$\sigma_{\phi,z_t} = \text{Softplus}\left(\text{MLP}\left(\text{Concat}\left(\widehat{z_{t-1}}, \widehat{x}_t,\ e_t,\ c_t\right)\right)\right) \tag{10}$$

$$z_t = \mu_{\phi,z_t} + \sigma_{\phi,z_t} \cdot \epsilon_t,\ \epsilon_t \sim \text{Uniform}(0,1) \tag{11}$$

where ϕ represents the parameters of the inference network. By concatenating the one-hot vector c_t, it can achieve the effect of switching [12] using MLP, which determines the parameter category (μ and σ) of the Gaussian distribution to which z_t belongs. Here we use the re-parameterization trick [24] of sampling from the posterior distribution of the latent space to obtain the specific posterior latent variable z_t to enable backpropagation. By simultaneously considering the four factors, GTGmVAE can model the temporal dependency of the posterior latent variable and the global temporal dependency of the input sequences to achieve strong temporal modeling capability and accurate posterior variational inference of the latent variable, with adaptability to different normal patterns.

B. Global Latent-Space Temporal Dependency Gaussian Mixture Generation Network

This section details the proposed new global latent-space temporal dependency Gaussian mixture generation network, which is used to generate the sequence x'_T. Combined with its probabilistic graphical model (Fig. 5(b)), we give the joint probability distribution of the generation network:

$$\begin{aligned} &p_\theta\left(x'_{1:T}, z_{1:T}, d_{z_{1:T}}, h_{1:T}, c_{1:T} | x_{1:T}, z_0, h_0\right) \\ &= p_\theta\left(x'_{1:T} | z_{1:T}, d_{z_{1:T}}, h_{0:T-1}\right) p_\theta\left(d_{z_{1:T}} | z_{1:T}\right) \\ &\quad \times p_\theta\left(z_{1:T} | h_{0:T-1}, c_{1:T}, z_0\right) p_\theta\left(c_{1:T}\right) p_\theta\left(h_{1:T} | x_{1:T}, h_0\right) \\ &= \prod_{t=1}^{T} p_\theta\left(x'_t | z_t, d_{z_t}, h_{t-1}\right) p_\theta\left(d_{z_t} | z_{1:T}\right) p_\theta\left(z_t | z_{t-1}, h_{t-1}, c_t\right) \\ &\quad \times p_\theta\left(c_t\right) p_\theta\left(h_t | h_{t-1}, x_t\right) \end{aligned} \tag{12}$$

where $p\left(c_t\right)$ is a Uniform distribution. In the following, we describe the computational process based on the structure of the generation network (Fig. 4(b)).

1) Prior of Latent Variable z: Since we introduce the indicator variable c and make the latent variable z dependent on c (§ IV-A.3), z follows a Gaussian mixture distribution:

$$p\left(z_t\right) = \sum_{k=1}^{K} \pi_k \mathcal{N}\left(z_t | \mu_{t,k}, diag\left(\sigma_{t,k}^2\right)\right) \tag{13}$$

where $\sum_{k=1}^{K} \pi_k = 1$. Combining the knowledge of the probabilistic graphical model (Fig. 5(b)) and Fig. 4(b), it is known that the prior of the current time latent variable z_t is calculated from three parts: (a) the nonlinear temporal dependency factor $\widehat{z_{t-1}}$ of previous time latent variable z_{t-1}, which is the same as § IV-A.4. (b) Historical factor h_{t-1}, which captures the historical temporal information of input sequence $\widehat{x}$ using LSTM, i.e., $h_t = \text{LSTM}\left(\widehat{x}_t, h_{t-1}\right)$. Note that we cannot model the global temporal dependency of $\widehat{x}$ here (the reconstruction of $\widehat{x}_t$ in § IV-B.2 is similar): the global temporal information contains information of $\widehat{x}_t$ itself, which causes the model to reconstruct $\widehat{x}_t$ using only $\widehat{x}_t$, and results in the KL divergence of the posterior and prior of z_t becoming 0, leading to model failure. (c) The current time indicator variable c_t, which indicates the category of the prior distribution of z_t, thus providing GTGmVAE with the adaptability to model different normal patterns. Taken together, the prior of z_t is calculated by:

$$\mu_{\theta,z_t} = \text{MLP}\left(\text{Concat}\left(\widehat{z_{t-1}}, h_{t-1},\ c_t\right)\right) \tag{14}$$

$$\sigma_{\theta,z_t} = \text{Softplus}\left(\text{MLP}\left(\text{Concat}\left(\widehat{z_{t-1}}, h_{t-1},\ c_t\right)\right)\right) \tag{15}$$

where θ represents the parameters of the generation network. The temporal modeling of $\widehat{x}$ and the prior z is also separated to avoid mutual influence. By simultaneously considering the three factors, GTGmVAE can accurately select the Gaussian distribution category via c_t using $\widehat{x}_t$ and its global temporal dependency, and integrate the historical temporal information of the latent variable and input sequences, thus achieving strong adaptability and temporal modeling capability.

2) Reconstruction of Input Sequence x: Combining the knowledge of the probabilistic graphical model (Fig. 5(b)) and Fig. 4(b), it is known that the reconstruction of the current time sample x'_t is generated from three parts: (a) Historical information h_{t-1} of the input sequence $\widehat{x}$. (b) The current time latent variable z_t. (c) The global temporal dependency factor d_{z_t} of z_t, which is obtained by the new Global Temporal Dependency Decoder (GTDD). Similar to GTDE, GTDD is stacked by several Transformer Encoder Blocks [28], and through the self-attention mechanism, it can effectively capture the global temporal dependency of z_t in sequence z with each other. The computational procedure of GTDD is as follows:

$$z_{input}^i = \begin{cases} z, & i = 1 \\ d_z^{i-1}, & i > 1 \end{cases} \tag{16}$$

$$a^i = \text{MultiHeadAttntion}\left(z_{input}^i, z_{input}^i, z_{input}^i\right) \tag{17}$$

$$a_{norm}^i = \text{LayerNorm}\left(a^i + z_{input}^i\right) \tag{18}$$

$$d_z^i = \text{LayerNorm}\left(\text{FeedFoward}\left(a_{norm}^i\right) + a_{norm}^i\right) \tag{19}$$

By cyclically performing the above computation N times, we take the final d_z^N as the global temporal dependency factor sequence $d_z = \{d_{z1}, d_{z2}, \cdots, d_{zT}\}$. By introducing d_z, the global temporal dependency of the latent space can be fully considered during reconstruction, which effectively improves the capability of GTGmVAE to generate temporal samples. Let x'_t follow a Gaussian distribution, we have:

$$x'_t \sim \mathcal{N}\left(\mu_{\theta,x'_t}, diag\left(\sigma^2_{\theta,x'_t}\right)\right) \tag{20}$$

$$\mu_{\theta,x'_t} = \text{DCNN}\left(\text{MLP}\left(\text{Concat}\left(z_t, d_{z_t}, h_{t-1}\right)\right)\right) \tag{21}$$

$$\sigma_{\theta,x'_t} = \text{Softplus}\left(\text{DCNN}\left(\text{MLP}\left(\text{Concat}\left(z_t, d_{z_t}, h_{t-1}\right)\right)\right)\right) \tag{22}$$

Here DCNN is a one-dimensional deconvolutional neural network that can help to better generate structured samples [12]. By simultaneously considering the three factors, GTGmVAE fully incorporates the global temporal dependency of the latent variable sequence z and the historical information of the input sequence $\widehat{x}$ when generating the reconstructed samples x'_t, thus achieving a strong temporal sample generation capability.

C. Model Training

According to the probabilistic graphical model (Fig. 5), the prior distribution of c is $p_\theta(c_t)$, the variational posterior distribution is $q_\phi(c_t|x_{1:T})$, the prior distribution of z is $p_\theta(z_t|z_{t-1}, x_{<t}, c_t)$, the variational posterior distribution is $q_\phi(z_t|z_{t-1}, x_{1:T}, c_t)$, the generation distribution of x is $p_\theta(x_t|z_{1:T}, x_{<t})$. Consistent with VAE, our goal is to maximize the log-likelihood of the observation sample x_t while minimizing the KL divergence between the variational posterior distribution $q_\phi(z_t, c_t|z_{t-1}, x_{1:T})$ and the prior distribution $p_\theta(z_t, c_t|z_{t-1}, x_{<t})$. Below is the loss function of GTGmVAE, the specific calculation of it is in **Supplementary Appendix**.

$$\begin{aligned}
&\mathcal{L}(\theta, \phi; x_t) \\
&= \mathbb{E}_{q_\phi(z_t, c_t|z_{t-1}, x_{1:T})}\left[\log p_\theta(x_t|z_{1:T}, x_{<t})\right] \\
&\quad - \text{KL}\left(q_\phi(z_t, c_t|z_{t-1}, x_{1:T}) \| p_\theta(z_t, c_t|z_{t-1}, x_{<t})\right) \\
&= \mathbb{E}_{q_\phi(z_t, c_t|z_{t-1}, x_{1:T})}\left[\log p_\theta(x_t|z_{1:T}, x_{<t})\right] \\
&\quad - \mathbb{E}_{q_\phi(c_t|x_{1:T})}\left[\text{KL}\left(q_\phi(z_t|z_{t-1}, x_{1:T}, c_t) \| p_\theta(z_t|z_{t-1}, x_{<t}, c_t)\right)\right] \\
&\quad - \text{KL}\left(q_\phi(c_t|x_{1:T}) \| p_\theta(c_t)\right)
\end{aligned} \tag{23}$$

D. Anomaly Detection

GTGmVAE learns the normal network patterns of EPs during training. Therefore, with the same assumption as in the previous work on anomaly detection, the more the input sample of an EP matches the normal network patterns of that EP, the higher the probability of its reconstruction. We use the reconstruction probability S_t of the sample x_t as its anomaly score to judge whether it is anomalous or not:

$$S_t = \log p_\theta(x_t|z_{1:T}, x_{<t}) \tag{24}$$

The sample x_t is considered anomalous if S_t falls below the specified threshold. Similar to previous work, we use POT [15], [30] to implement automatic threshold selection.

E. Characteristics and Advantages of GTGmVAE

1) Strong Feature Extraction and Integration Capability: Through the proposed new LGFE, GTGmVAE can effectively combine global and local perspectives to extract and integrate the features of input samples, which can help the model to learn the distribution to which the input samples belong more efficiently, thus contributing to the subsequent latent variable inference and temporal dependency modeling, as well as effectively enhancing the representation capability of GTGmVAE to achieve the desired adaptability.

VAE [24] and its dynamic variants (e.g., VRNN [14]) typically compress the dimension of input samples through fully connection networks, resulting in a large amount of implicit structural information being discarded, which negatively affects subsequent latent variable inference and temporal dependency modeling. Some advanced work [12], [27] uses 1D CNN to extract features from the input samples. However, this approach is still insufficient: due to the limitation of convolutional operations, it can only process the local features, and thus remains unable to fully extract and integrate the implicit structural information. In contrast, the LGFE included in GTGmVAE achieves a strong feature extraction and integration capability by serially combining the 1D CNN and the Transformer Encoder, first extracting the low-level implicit structural features of the input samples from a local perspective, and then further extracting and integrating the high-level features of the input samples from a global perspective.

2) Strong Temporal Modeling Capability: In the inference network, GTGmVAE accurately infers the latent variables based on the global temporal dependency of input sequences, and the temporal modeling of latent variables and input sequences are performed separately to avoid the negative effects due to the interaction between stochastic latent variables and deterministic input sequences. In the generation network, GTGmVAE improves the ability of temporal sample generation by combining the global temporal dependency of latent variables with the historical information of input sample sequences.

Existing representative VAE-based time-series anomaly detection methods [12], [14], [15], [16], [19] typically model the temporal dependency of input sequences via RNN or LSTM, but this can only model the historical temporal information, ignores the global temporal dependency, and performs poorly on long sequences. Meanwhile, in the above methods, LSTM-VAE [19] does not model the temporal dependency of stochastic latent variables and ignores the temporal dependency of potential representations of the input sequences, resulting in poor performance [16]. While the other methods model the temporal dependency of stochastic latent variables, they use the same RNN or LSTM to process both stochastic latent variables and deterministic input sequences simultaneously, leading to negative effects. Finally, none of the above methods further consider the temporal dependency of latent variables in the generation stage, which is very effective in improving the model's ability to generate temporal samples.

Compared to VAE-based time-series anomaly detection models, Transformer-based time-series anomaly detection models [13], [17], [21], [22] use Transformer [28] to model the input sequence, effectively capturing its global temporal dependency. However, Transformer-based methods are

typically deterministic and cannot model the stochasticity of the data like VAE-based methods, and thus lack the robustness to cope with uncertainty in the time-series data, which can affect the accuracy of the anomaly detection, especially considering that VCNs have complex temporal non-stationarity.

We can see that GTGmVAE combines the advantages of both VAE-based and Transformer-based methods: it both considers the temporal modeling of stochastic variables and captures the global temporal dependency using Transformer. In addition, GTGmVAE effectively solves the above problems faced by current time-series anomaly detection models through posterior inference based on the global temporal dependency of input sequences, separated temporal modeling of deterministic and stochastic variables, and generation of temporal samples combining the global temporal dependency of latent variables. Therefore, compared to existing methods, GTGmVAE can effectively model the complex temporal dependency and non-stationarity of VCNs to cope with **C3**.

3) High Adaptability: Current time-series anomaly detection models [14], [15], [16], [19] typically make the latent variable obey a single diagonal Gaussian distribution, which limits their adaptability to learn different normal patterns. GmVAE [20] and SGmVRNN [12] achieve "one-to-many" adaptability to learn different normal patterns by making the latent variable obey a Gaussian mixture distribution, and introducing the indicator variable to denote the distribution category of the latent variable corresponding to the different normal patterns. However, they either neglect to capture the temporal dependency of input sequences [20], or rely on only the historical information rather than the global temporal information to determine the category of the indicator variable [12]. Given that VCNs contain multiple EPs with different patterns and have temporal non-stationarity, these methods ignore the dependencies between the input sample and other samples in the sequence, and thus fail to accurately capture the dynamic change in the category of the indicator variable.

In contrast, GTGmVAE not only introduces the Gaussian mixture distributed latent variable, but also accurately infers the category of the indicator variable based on the global temporal dependency of input sequences, thus accurately capturing the dynamic changes in the category of the indicator variable. Combined with the expressiveness of LGFE and strong temporal modelling capability, GTGmVAE is able to model the normal patterns of different EPs simultaneously, as well as multiple different normal patterns contained in one EP due to temporal non-stationarity, to effectively cope with **C2**.

V. Evaluation

This section provides a detailed evaluation of the proposed anomaly detection model GTGmVAE and the feature model MFM. First, we describe the experimental setup. Second, we compare GTGmVAE with nine representative state-of-the-art models to validate its performance. Third, we conduct comprehensive ablation experiments to demonstrate the effectiveness of each component in GTGmVAE and the effectiveness of each level features in MFM. Forth, we evaluate the online detection efficiency of the proposed models. Finally, the experiments of hyper-parameters are in **Supplementary Appendix**.

A. Experimental Setup

1) Datasets: Currently, there are two representative public VCN anomaly detection datasets: CIC-IDS2018 [23] and ISOT-CID [18]. CIC-IDS2018 was constructed by building a VCN on AWS and collecting 10 days of traffic for each EP. According to the official records, there are 14 EPs (see **Supplementary Appendix** for details) that suffered attacks, i.e. have anomalous samples, and thus only these 14 EPs can be used for evaluation. CIC-IDS2018 covers the main types of attacks faced by VCNs, including external north-south attacks against EPs, and internal east-west lateral movement attacks launched by compromised EPs against other EPs. We cannot use ISOT-CID for evaluation for the following reasons: 1) ISOT-CID has two phases. The traffic collection of *Phase1* uses random sampling, where each collection randomly lasts from 2 s to 12 s and is repeated every 80 s. This disrupts the temporal continuity of the original traffic, making the model unable to perform temporal modeling; 2) The number of available EPs in *Phase2* is only 5, which is insufficient to evaluate model's adaptability. More importantly, *phase2* lacks the main attacks faced by VCNs such as external north-south DDoS and internal east-west attacks. In addition, to further validate GTGmVAE, we also evaluated it on a time-series anomaly detection benchmark dataset SMD [15]. Despite being orientated towards a different scenario, SMD has similar properties to VCN anomaly detection: it contains time-series KPI data of 28 different servers and can be used to evaluate the model's adaptability and temporal modeling capability.

We use the proposed MFM to construct continuous samples based on the fixed and disjoint time window for the 14 EPs in CIC-IDS2018, with a dimension of 91 and a time window size of 3 s. For each EP's samples in a day, its first anomalous sample appears basically near 50% of all samples. Therefore, all continuous samples before the first anomalous sample are used for training. If the first anomalous sample is located in the last 50% samples in a day, the first 50% samples are used for training. The remaining samples of the day are used for test. For each EP, its 10-day training and test samples are spliced separately. Therefore, the length of training sets after division is less than the length of test sets, and they all have temporal continuity. To address **C2**, we use "one-to-many" strategy, i.e., splice the samples from all 14 EPs in CIC-IDS2018 to form a training set, train a model, and evaluate it on test sets of each EPs. SMD is treated similarly using 'one-to-many" strategy.

2) Evaluation Metrics: We use the most commonly used metrics for unsupervised time-series anomaly detection, including Precision, Recall and F1-score. F1-score balances Precision and Recall and is currently the most concerned metric for unsupervised time-series anomaly detection. We follow the point adjustment approach [31] taken by the vast majority of unsupervised time-series anomaly detection methods, i.e., an anomalous segment is considered to be correctly detected if any one of anomalies in it is detected. All experiments are one-off effort and repeated 10 times to report the average metrics.

3) Compared Methods: We compared nine representative state-of-the-art unsupervised anomaly detection methods: 1) VRNN [14]: extends VAE using RNN to make it capable of temporal modeling. 2) GmVAE [20]: improves the adaptability of VAE by making the latent variable follow a Gaussian mixture distribution. 3) OmniAnomaly [15]: a robust

stochastic RNN-based time-series anomaly detection model. 4) SDFVAE [16]: a robust static and dynamic factorized VAE for time-series anomaly detection. 5) TranAD [17]: a time-series anomaly detection model based on Transformer and adversarial training. 6) Anomaly Transformer [21]: a time-series anomaly detection model based on Transformer and association discrepancy. 7) SGmVRNN [12]: a time-series anomaly detection model combining VRNN and GmVAE with temporal modeling capability and adaptability. 8) PUAD [13]: a time-series anomaly detection model combining Transformer and prototype learning with temporal modeling capability and adaptability. 9) DCDetector [22]: a time-series anomaly detection model based on Transformer and contrastive learning.

The above methods are not only the recent state-of-the-art, but also representative in feature extraction capability, temporal modelling capability and "one-to-many" adaptability. (a) For feature extraction capability, SGmVRNN and SDFVAE use CNN to replace the fully connection networks in VAE for dimension compression and feature extraction, which can extract and integrate the implicit features of input samples from a local perspective. In addition, to further improve the representativeness of the compared methods, the implementations of VRNN and GmVAE adopt the SGmVRNN approach, i.e., 1D CNN is used to improve performance. (b) For temporal modelling capability, all of the above methods except GmVAE have good temporal modelling capability. Among them, VRNN, SDFVAE, OmniAnomaly and SGmVRNN are the current representative VAE-based time-series anomaly detection models that can adequately represent the stochastic nature of time-series data; while TranAD, Anomaly Transformer, PUAD and DCDetector are the current representative Transformer-based time-series anomaly detection models. Compared to the VAE-based methods, they can better model temporal dependency from a global perspective and can better process long sequences. (c) For adaptability, GmVAE, SGmVRNN and PUAD are the representative methods that make the model obtain "one-to-many" adaptability by making the latent variables obey the Gaussian mixture distribution and introducing the prototype learning, respectively. Also, Anomaly Transformer and DCDetector find the discrepancies between the sample and others in the sequence rather than just learning normal patterns, thus achieving a degree of adaptability along with the temporal modeling capability. In summary, these nine methods are both advanced and representative, and are therefore able to comprehensively demonstrate the advantages of GTGmVAE.

4) Hyper-Parameters: Since CIC-IDS2018 and SMD differ in the sample dimensions (91 vs 38), the parameters of LGFE are different: For CIC-IDS2018, (k_i, s_i, c_i) are $(3, 3, 8)$, $(3, 3, 16)$, $(2, 2, 32)$, the number of Transformer Encoder Blocks is 1 with heads of 2, d_k and d_v of 16. For SMD, (k_i, s_i, c_i) are $(2, 2, 8)$, $(2, 2, 16)$, $(2, 2, 32)$, the number of Transformer Encoder Blocks is 2 with heads of 2, d_k and d_v of 16. The output dimension of LGFE is 20. For GTDE and GTDD, the number of Transformer Encoder Blocks is 6 with heads of 8, d_k and d_v of 64. The dimension d of the latent variable z is 10. The dimension (i.e., K) of the indicator variable c is 5. The Adam optimizer is used with learning rate of 0.0002. The number of epochs is 20, the batch size is 512, and the sequence length T is 20. Gumbel-Softmax has an initial temperature λ of 5.0 and a rate of decrease of 0.1 per epoch up to a minimum value of 0.1. For SMD, the probability p associated with the initial threshold used in POT is set to 0.003. For CIC-IDS2018, as the percentage of anomalous samples varies considerably between EPs, the value of p is 0.15 for EP1, 0.07 for EP2, 0.01 for EP4 and 0.015 for EP3 and EP5-EP14. We present further experiments and discussion of the main hyper-parameters in the **Supplementary Appendix**. For GTGmVAE and all comparison methods, we uniformly use the model of the last epoch for test. This removes setup bias and ensures fairness of comparisons. Meanwhile, in "one-to-many" training, we cannot find a uniform epoch for N EPs: the optimal training epoch may be different for each EP. Also, this approach is the uniform and standard approach adopted by SGmVRNN [12], which proposed the "one-to-many" method. For GTGmVAE and all comparison methods, we use POT with uniform parameters to determine thresholds on the test set. Typically, POT thresholds are determined on the training set [15]. However, in "one-to-many" task, POT thresholds are determined on the test set [12]. This is because in "one-to-many" task, we splice the training data of multiple EPs and evaluate the test set of each EP separately. Therefore, we cannot determine a uniform threshold or determine thresholds for each EP separately on such a training set. Meanwhile, this approach does not rely on labels, does not affect the trained model, and is able to adaptively adjust the threshold according to the dynamic changes in distributions of the current sample, thus better adapting to the temporal non-stationarity of VCNs.

5) Experimental Platform: All experiments are conducted on a server with Intel Xeon Gold 5120 * 2 CPU, 128 GB RAM and Nvidia Tesla T4 * 2 GPU. MFM and GTGmVAE are implemented using Python 3.7.10 and PyTorch 1.9.0.

TABLE IV
RESULTS OF COMPARISON OF GTGMVAE WITH NINE REPRESENTATIVE STATE-OF-THE-ART METHODS

Methods	CIC-IDS2018			SMD		
	Precision	Recall	F1-score	Precision	Recall	F1-score
VRNN	0.9158	0.8327	0.8617	0.9718	0.8818	0.9246
GmVAE	0.9117	0.8092	0.8520	0.9691	0.8262	0.8914
OmniAnomaly	0.8955	0.7127	0.7932	0.9731	0.7753	0.8629
SDFVAE	0.9162	0.8659	0.8891	0.9712	0.8713	0.9185
TranAD	0.9092	0.7298	0.8097	0.9772	0.8612	0.9155
Ano. Tran.	0.9154	0.9370	0.9255	0.9726	0.8733	0.9199
SGmVRNN	0.9188	0.8928	0.9042	0.9737	0.8961	0.9332
PUAD	0.9259	0.9440	0.9342	0.9768	0.8594	0.9143
DCDetector	0.8925	0.9473	0.9191	0.7223	0.9091	0.8043
GTGmVAE	0.9262	0.9683	**0.9465**	0.9742	0.9104	**0.9412**

B. Comparison Experiments

The comparison results of GTGmVAE with nine representative state-of-the-art methods are shown in table IV. The "one-to-many" strategy is used for all methods. As the main metric, the highest F1 score of all methods is shown in bold.

GTGmVAE achieves the highest F1-score on both CIC-IDS2018 and SMD. For CIC-IDS2018, GTGmVAE combined with MFM achieves the highest and desirable F1 score (0.9465), fully demonstrating the effectiveness of GTGmVAE and MFM in VCN anomaly detection, and the advantages of GTGmVAE in temporal modeling and adaptability. Therefore,

TABLE V
RESULTS OF THE ABLATION EXPERIMENTS ON GTGMVAE

Methods	CIC-IDS2018			SMD		
	Precision	Recall	F1-score	Precision	Recall	F1-score
Type1	0.9271	0.9523	0.9387	0.9724	0.8870	0.9277
Type2	0.9207	0.9258	0.9219	0.9736	0.8974	0.9339
Type3	0.9225	0.9331	0.9274	0.9748	0.9012	0.9365
Type4	0.9235	0.9463	0.9325	0.9735	0.9060	0.9385
Type5	0.9240	0.9366	0.9263	0.9743	0.9024	0.9369
Type6	0.9220	0.8927	0.9066	0.9706	0.8385	0.8996
GTGmVAE	0.9262	0.9683	**0.9465**	0.9742	0.9104	**0.9412**

MFM and GTGmVAE can effectively address **C1**, **C2** and **C3**. For SMD, GTGmVAE achieves the highest F1-score (0.9412). Considering that SMD is one of the most commonly used benchmark datasets for time-series anomaly detection with similar properties to VCN anomaly detection, this fully demonstrates the advantages of GTGmVAE in temporal modeling and adaptability, and its robustness to different tasks.

The following is a specific analysis of the methods compared. Of the methods compared, all except GmVAE have the temporal modeling capability. The importance of considering temporal modeling in **C3** is well demonstrated by the comparison with GmVAE. In contrast, the comparison with methods other than GmVAE demonstrates the advantages of GTGmVAE in terms of the temporal modeling capability and the adaptability. In addition, SGmVRNN, PUAD, Anomaly Transformer and DCDetector all have both the temporal modeling capability and the adaptability to different normal patterns. However, their adaptability and temporal modeling capabilities are insufficient to meet the requirements of VCN anomaly detection. By comparing with these methods, it is well demonstrated that GTGmVAE not only has the better adaptability to address **C2**, but also has the better temporal modeling capability to effectively model the complex temporal dependency of VCNs and more effectively capture and model the temporal non-stationarity to address **C3**.

C. Ablation Study for Detection Model GTGmVAE

To validate each component in GTGmVAE, we conduct the following extensive ablation experiments. Type1: Replace the latent variable distribution from a Gaussian mixture distribution to a single Gaussian distribution. Type2: Replace LGFE with full connections. Type3: Replace LGFE with 1D CNN. Type4: Remove the modeling of the global temporal dependency of latent variables in generation network. Type5: Further replace the modeling of the global temporal dependency with the modeling of the historical temporal dependency on Type4. Type6: Disregard the modeling of the temporal dependency.

Type1 is used to validate the effect of using the Gaussian mixture distribution on model's adaptability. Type2-3 are used to validate the proposed LGFE. Type4-6 are used to validate the proposed optimizations of GTGmVAE in temporal modeling. The results of the ablation study are shown in table V.

The performance of Type1 is degraded due to its reduced adaptability. Note that besides using the Gaussian mixture distribution, GTGmVAE also improves the adaptability through LGFE (improves the model's representation capability) and the global temporal dependency modeling (improves the model's capability to model complex temporal dependencies), so Type1 still has a degree of adaptability and its performance is only slightly degraded. The fully connection in Type2 is used in VAE and its variants (e.g., VRNN), which just simply reduces the input's dimension, losing the implicit feature information. The 1D CNN in Type3 comes from advanced methods [12], [27] and has a better feature extraction capability, thus slightly improving the performance compared to Type2. However, compared to the proposed LGFE, 1D CNN only processes the local features and ignores the global features. In addition, both Type2 and Type3 reduce the model's representation capability and therefore its adaptability. As a result, their performance is both degraded. The capability to generate temporal samples in Type4 is reduced, which degrades the performance. Type5 further replaces all the global temporal dependency modeling with the historical temporal dependency modeling on Type4, reducing both the inference and generation capabilities and further degrading the performance. This demonstrates the advantages of the global temporal dependency modeling for the inference of the indicator and latent variables and the generation of temporal samples. The performance is significantly degraded in Type6 as no temporal modeling is considered at all. This demonstrates the importance of modeling the complex temporal dependency of VCNs highlighted in **C3**.

TABLE VI
RESULTS OF THE ABLATION EXPERIMENTS ON MFM

Feature Model	CIC-IDS2018		
	Precision	Recall	F1-score
Aldribi et al. [18]	0.9120	0.8420	0.8743
Level 1	0.9168	0.8861	0.9009
Level 1 + Level 2	0.9253	0.8983	0.9116
MFM (Levels 1, 2 and 3)	0.9262	0.9683	**0.9465**

TABLE VII
PERFORMANCE COMPARISON OF DIFFERENT FEATURE MODELS ON EP3

Feature Model	EP3 in CIC-IDS2018		
	Precision	Recall	F1-score
Aldribi et al. [18]	0.6653	0.4338	0.5243
Level 1	0.9143	0.7166	0.8034

TABLE VIII
PERFORMANCE COMPARISON OF DIFFERENT FEATURE MODELS ON EP4

Feature Model	EP4 in CIC-IDS2018		
	Precision	Recall	F1-score
Aldribi et al. [18]	0.6370	0.3674	0.4660
Level 1	0.8773	0.7271	0.7951

D. Ablation Study for Feature Model MFM

To validate the effectiveness of each level features in MFM, we uniformly conduct ablation experiments using GTGmVAE for each level features, and the results are shown in table VI.

The feature model of Aldribi et al. [18] is representative in VCN anomaly detection task and is used to construct Level 1

TABLE IX
PERFORMANCE COMPARISON OF DIFFERENT FEATURE MODELS ON EP2

Feature Model	EP2 in CIC-IDS2018		
	Precision	Recall	F1-score
Level 1 + Level 2	0.8275	0.2750	0.4128
MFM (Levels 1, 2 and 3)	0.9462	0.9773	0.9615

TABLE X
DETECTION LATENCY FOR DIFFERENT SIZED BATCH

Number of Samples	Detection Latency (s)
Batch size = 1	0.0468
Batch size = 512	0.1958
Batch size = 1024	0.3636

features in MFM. However, since only the packet frequency is considered, this feature model cannot comprehensively characterize the basic network patterns of VCNs, resulting in poor detection performance. Meanwhile, despite designing the egress entropy features, Aldribi et al. only consider the ingress direction while ignoring the egress entropy features when actually constructing feature samples for the detection model, and therefore ignore characterizing the randomness of the traffic sent out from EPs, which we experimentally demonstrate to be important for detecting attacks launched by compromised EPs. Specifically, we use EP3 and EP4 in CIC-IDS2018 as examples for our study. After being compromised, EP3 and EP4 launch port scanning attacks to other normal EPs in the same network segment (usually as the first step of the lateral movement attack), randomly probing for ports that could be used for the attack. Since the feature model of Aldribi et al. ignores entropy features in the egress direction to characterizing the randomness of the traffic sent out from EPs, it is difficult to distinguish such attacks, and thus the F1-scores for EP3 and EP4 are 0.5243 and 0.4660, respectively (as shown in table VII and table VIII). In contrast, we take into account the egress entropy features when constructing Level 1 features, so that Level 1 features provide better detection performance than Aldribi et al., which results from being able to more effectively detect lateral movement attacks from compromised EPs. As shown in table VII and table VIII, the F1-scores for EP3 and EP4 are significantly higher when using Level 1 features, and are 0.8034 and 0.7951, respectively.

Further, due to the more comprehensive characterization of the basic network patterns of VCNs from both the throughput and frequency perspectives, Level 1 and 2 features provide better performance than Level 1 features alone. This is mainly due to the better differentiation of anomalous samples and generation of normal samples through the more comprehensive characterization, resulting in an improvement of 0.0122 in Recall and 0.0085 in Precision, leading to a higher F1-score.

Finally, the complete MFM incorporates key topology features on top of Level 1 and 2 features to comprehensively and accurately characterize the network patterns of VCNs by considering the uniqueness of VCNs. CIC-IDS2018 covers the main types of attacks faced by VCNs, including attacks launched externally against internal EPs, and lateral movement attacks launched from internal compromised EPs. As described in § III-C, these attacks will result in significant changes to the stable normal patterns of the EP's connection topology graph. Specifically, we take EP2 as an example for our study. EP2 suffers from two typical DDoS attacks: LOIC (Low Orbit Ion Cannon) and HOIC (High Orbit Ion Cannon) [23]. Among them, LOIC mainly sends a large number of TCP, UDP and HTTP requests to overwhelm the target EP, but usually relies on a single client to launch the single attack. HOIC, on the other hand, has greater stealth and attack capability due to its multi-threading, scripting support and automation, and thus is more difficult to detect. However, both attacks lead to significant changes in the connection topology graph of the EP (mainly in malicious external nodes and edges). Therefore, by effectively characterizing the connection topology graph of EP, the complete MFM with the addition of Level 3 features can effectively distinguish the above attacks compared to Level 1+ Level 2 features. As shown in table IX, the F1-score for EP2 is significantly improved from 0.4128 to 0.9615. Meanwhile, from an overall perspective, as shown in table VI, the incorporation of topology features in MFM leads to a significant performance improvement over Level 1 and 2 features, especially in terms of the better differentiation of anomalous samples, resulting in an improvement of 0.07 in Recall, leading to a higher F1-score. This confirms the validity of our observation, analysis and derived insight of the uniqueness of VCNs. The comparison of the four experiments fully demonstrates the effectiveness of MFM in addressing **C1** and the effectiveness of each level of features in MFM.

E. Efficiency and Scalability of Online Detection

In this section, we evaluate the efficiency and scalability of the real-time online detection of GTGmVAE, and the efficiency of the feature computation for MFM.

As we use the "one-to-many" strategy, a trained model can be used for online detection of multiple different EPs simultaneously. Given the parallelization nature of dedicated hardware GPUs mainly used for the model inference, online detection of multiple EPs can be performed in parallel by combining their samples into the same *Batch*, which greatly improves the scalability. For example, for 512 EPs, we can load a trained model using only one GPU, feed the samples of these EPs as a *Batch* of size 512 and perform inference. In contrast, the "one-to-one" strategy requires 512 exclusive models to be trained for 512 EPs and fed through 512 GPUs for inference, and each model can only be fed one sample at a time, which is obviously inefficient and wasteful. Under the "one-to-many" strategy, we evaluate the inference efficiency for three scenarios: 1) one sample at a time (*Batch* size = 1); 2) 512 samples at a time (*Batch* size = 512); 3) 1024 samples at a time (*Batch* size = 1024). The results are shown in table X. Even with the consideration of the scalability for practical detection, the detection latency of GTGmVAE is still very low even when 512/1024 EPs are detected simultaneously (0.1958 s and 0.3636 s), which can clearly meet the requirement for real-time online detection with the time window size of 3 s.

Besides the inference efficiency of GTGmVAE, for real-time detection, we also need to focus on the feature computation efficiency of MFM. In this paper, we use Python to implement feature computation for CIC-IDS2018 in a single-threaded

manner. Since no flow states need to be maintained and only a few packet header fields need to be counted, the average time to compute features for constructing a sample is 0.0233 s. Therefore, with the time window size of 3 s, our detection model GTGmVAE and feature model MFM can clearly meet the requirement for real-time online detection. In addition, MFM can easily compute features directly using the statistics from the cloud built-in virtual switch [32] (§ III-D). Therefore, implementing feature computation inside the cloud platform will maximize the efficiency. We make it our future work.

VI. Discussion

A. Discussions on MFM

1) Flexibility and Challenges of Deploying MFM in VCNs: The traffic statistics required for MFM are available in different ways in VCNs. If a virtual switch with statistics capabilities is built into the cloud (for example, the cloud virtual switch VFP [32] directly provides statistics on flow tuples, packet frequency and throughput), MFM can directly utilize these statistics. Even if such switches are not deployed in the cloud, in a VM cloud, we can develop plugins on the host's virtual switch (e.g., Open vSwitch [33]) to obtain the required packet information, while in a container cloud, the required packet information can be captured by plugging the eBPF program into the host. The above approaches do not require the deployment of specialized network middleware and the traffic redirection, and do not lead to significant overheads compared to traditional stateful flow processing. In summary, the core idea of MFM is to characterize VCNs in a non-stateful manner using only simple packet header information, and thus it is very flexible: the information required for MFM can be obtained in different types of clouds, either through virtual switches or eBPF programs. In addition, MFM is essentially a theoretical contribution, whereas in actual large-scale clouds, it may face deployment challenges such as interface compatibility of virtual switches, and architectural design for distributed implementations, etc. In future studies, we aim to validate and optimize the implementation details of MFM in real clouds (with or without the switches that natively support statistical functions), and assess the possible deployment challenges and system overheads at different scales.

2) MFM Vs. End-to-End Methods: Although end-to-end methods have become an important trend in deep learning, however, manually designed MFMs still have the following advantages over end-to-end methods where the raw traffic data is fed directly into a deep model: (a) Targeted extraction of key features: VCN traffic has complex temporal dependencies and needs to be characterized in multiple dimensions (e.g. frequency, throughput, topology, etc.). These key features are implicit in the raw data and have temporal relationships. In addition, VCN traffic contains large redundant data (noise). End-to-end methods are difficult to capture key features and are susceptible to noise interference, while manually designed MFMs can target and filter the most discriminative key features from multiple dimensions. (b) Real-time detection and computational overhead: Anomaly detection systems require fast decisions. End-to-end methods can incur a large computational burden due to redundancy and noise in raw data, affecting the real-time and scalability of detection. In contrast, manually designed MFM effectively reduces computational overhead and improves real-time detection through fixed time windows and non-stateful feature computation.

B. Discussions on GTGmVAE

1) Coarse-Grained Detection: Given the wide variety of attacks and unknown new attacks faced by VCNs, unsupervised methods are more suitable than supervised methods. However, as with typical unsupervised anomaly detection models, GTGmVAE can only detect anomalous behavior that deviates from the normal pattern and cannot make further judgments about the anomalies, such as the type of attack. Combined with the fixed time-window design of MFM, GTGmVAE is capable of fast coarse-grained anomaly detection and is suitable for use in alert systems or preliminary screening phases, and needs to be complemented with other methods (e.g., classification models) to make further judgements on anomalies.

2) Interpretability Challenges: GTGmVAE introduces Gaussian mixture distributed latent variables and models global temporal dependencies of input and latent variable sequences, which improves the model's adaptability and temporal modelling capability. However, this also leads to challenges for typical interpretability methods (e.g., LIME [34], SHAP [35]) in explaining the internal mechanisms of GTGmVAE, e.g., the diversity of the Gaussian mixture distribution makes local modelling difficult; and the complex long-range nonlinear relationships of time-series data make it difficult for these methods to capture. Therefore, it is necessary to consider improving the interpretability methods for VCN anomaly detection, such as sampling different distributions and slicing the input sequences into time series, to cope with these challenges.

3) Dataset Limitation: Due to dataset limitation, we can currently only evaluate the "one-to-many" adaptability of GTGmVAE on CIC-IDS2018, which contains 14 EPs. However, despite the small number of EPs, the effectiveness of GTGmVAE in improving the scalability can be validated: GTGmVAE that considers adaptability achieves higher performance compared to other representative state-of-the-art methods. Meanwhile, in the ablation experiments, models without the design of adaptability (Type1, 2 and 3) all perform worse than GTGmVAE. We also conduct additional experiments on the SMD dataset with more servers (28 in total) to further validate the scalability of GTGmVAE on a larger dataset. Finally, a key design of GTGmVAE to improve scalability is to make the latent variable obey a Gaussian mixture distribution, which intuitively reflects how the model learns different patterns by categorizing the input samples and corresponding to different Gaussian distribution components of the latent space. Theoretically, in VCNs with more EPs, we can adjust the number of categories of the Gaussian mixture distribution to improve the scalability. In the future, we will try to evaluate the scalability of GTGmVAE in a larger scale scenario.

VII. Related Work

A. Feature Model

1) Flow-Based Feature Model: Representative flow-based feature models include NSL-KDD [8], NB15 [9], and

CICFlowMeter [10]. NSL-KDD and NB15 are classical intrusion datasets with self-contained flow-based feature models. While CICFlowMeter is the latest flow-based feature model and contains more and finer-grained flow-level features. However, as mentioned in **C1**, these feature models cannot cope with the scalability challenge and ignore the uniqueness of VCNs. In contrast, MFM addresses the scalability challenge through non-stateful feature computation and combines multilevel features to comprehensively and accurately characterize VCNs.

2) Time-Window Based Feature Model: The feature model in Kitsune [7] is similar to Level 1 and 2 features in MFM and can meet the scalability requirements of **C1**. However, it lacks the consideration for VCNs and thus is unsuitable for VCN anomaly detection. Aldribi et al. [18] proposed a representative feature model for VCN anomaly detection. However, as described in § III-B and § III-C, it only considers the packet frequency and ignores the uniqueness of VCNs. Compared to the above methods, MFM characterizes VCNs more comprehensively and accurately by introducing topology features that consider the uniqueness of VCNs. PrivateEye [2] proposed a feature model for detecting compromised cloud VMs. However, it is designed for supervised classification and mainly characterizes the malicious behaviors of compromised VMs, introducing the prior knowledge of attacks. Therefore, it is not effective when used for unsupervised anomaly detection, which focuses on learning normal patterns rather than recognizing predefined attacks. In addition, PrivateEye argues that the compromised VMs differ from normal VMs in their connection patterns, which shares commonalities with our insight into the uniqueness of VCNs. However, we provide a more generalizable insight based on the property that the east-west traffic that dominates VCNs comes from the stable business logic: Any malicious behavior, including attacks from both external and internal sources against normal EPs (VMs or containers), and attacks launched by the compromised EPs against other EPs, can cause the connection patterns of these normal or compromised EPs to deviate from the baseline, while PrivateEye only considers compromised VMs. In contrast, MFM has better generality and can be better applied to unsupervised anomaly detection.

B. Detection Model

Kitsune [7] is a representative unsupervised intrusion detection method in resource-constrained environments (e.g., IoT) using a set of auto-encoders. PrivateEye [2], SCAE [5] and Aldribi et al. [18] are representative VCN intrusion detection methods using stacked contractive autoencoders with SVM, random forest and change point detection, respectively. However, they are either supervised [2], [5] or lack the necessary temporal modeling capability and adaptability [7], [18], making them difficult to apply to VCN anomaly detection. In contrast, GTGmVAE is unsupervised and has strong temporal modelling capability and adaptability obtained via global temporal dependency modelling and Gaussian mixture distributed latent space, which can cope with **C2** and **C3**.

Recently, unsupervised time-series anomaly detection models have been widely studied, which have the temporal modeling capability for the time-series data to achieve good detection performance. We have introduced recent representative state-of-the-art methods [12], [13], [14], [15], [16], [17], [21], [22] in § V-A. Among them, VRNN [14], OmniAnomaly [15], SDFVAE [16], and SGmVRNN [12] achieve temporal modeling capabilities through recurrent structures (e.g., RNN) on VAE, while PUAD [13], TranAD [17], Anomaly Transformer [21] and DCDetector [22] achieve time-series anomaly detection by combining Transformer with adversarial training, contrastive learning, and prototype learning. In addition, SGmVRNN, PUAD, Anomaly Transformer and DCDetector have the adaptability to different normal patterns besides the temporal modeling capability. In § IV-E, we analyze these methods in feature extraction capability, temporal modelling capability, and adaptability, and discuss the differences and advantages of GTGmVAE over them. Compared to the proposed GTGmVAE, these models either lack the sufficient adaptability or lack the sufficient temporal modeling capability to effectively cope with **C2** and **C3**, and thus perform poorly in VCN anomaly detection.

VIII. Conclusion

In this paper, we propose a new multilevel feature model MFM and a new unsupervised time-series anomaly detection model GTGmVAE for VCN anomaly detection. Of these, MFM incorporates the topology features that are specifically designed for the uniqueness of VCNs and the packet frequency and throughput features that characterize the basic network patterns of VCNs, and is therefore able to comprehensively and accurately characterize VCNs. GTGmVAE effectively improves the representation capability through a new local-global feature extractor, achieves the strong temporal modeling capability by modeling the global temporal dependency of the input samples and latent variables, and combines the Gaussian mixture distributed latent space with its strong temporal modeling capability and representation capability to achieve the strong adaptability, therefore being able to learn the normal patterns of multiple EPs simultaneously and to effectively address the complex temporal dependency and non-stationarity of VCNs. Extensive experiments on the VCN anomaly detection dataset CIC-IDS2018 and the time-series anomaly detection benchmark dataset SMD show that GTGmVAE with MFM achieves the desirable performance, and GTGmVAE outperforms all nine representative state-of-the-art detection models. For future work, we consider evaluating the challenges of deploying MFM in different types of clouds and evaluating GTGmVAE in larger scale scenarios.

References

[1] *The Nist Definition of Cloud Computing*, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, 2011, doi: 10.6028/nist.sp.800-145.

[2] B. Arzani et al., "PrivateEye: Scalable and privacy-preserving compromise detection in the cloud," in *Proc. 17th USENIX Symp. Networked Syst. Design Implement. (NSDI)*, Feb. 2020, pp. 797–815.

[3] B. I. Santoso, M. R. S. Idrus, and I. P. Gunawan, "Designing network intrusion and detection system using signature-based method for protecting OpenStack private cloud," in *Proc. 6th Int. Annu. Eng. Seminar (InAES)*, Aug. 2016, pp. 61–66.

[4] C. Mazzariello, R. Bifulco, and R. Canonico, "Integrating a network IDS into an open source cloud computing environment," in *Proc. 6th Int. Conf. Inf. Assurance Secur.*, Aug. 2010, pp. 265–270.

[5] W. Wang, X. Du, D. Shan, R. Qin, and N. Wang, "Cloud intrusion detection method based on stacked contractive auto-encoder and support vector machine," *IEEE Trans. Cloud Comput.*, vol. 10, no. 3, pp. 1634–1646, Jul. 2022.

[6] Cisco. (2016). *Cisco Global Cloud Index: Forecast and Methodology*. [Online]. Available: https://virtualization.network/Resources/Whitepapers/0b75cf2e-0c53-4891-918e-b542a5d364c5_white-paper-c11-738085.pdf
[7] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2018, pp. 1–15.
[8] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE Symp. Comput. Intell. Secur. Defense Appl.*, Jul. 2009, pp. 1–6.
[9] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. Mil. Commun. Inf. Syst. Conf. (MilCIS)*, Nov. 2015, pp. 1–6.
[10] Can. Inst. for Cybersecurity. (2022).*Cicflowmeter*. [Online]. Available: https://www.unb.ca/cic/research/applications.html
[11] T. Dargahi, A. Caponi, M. Ambrosin, G. Bianchi, and M. Conti, "A survey on the security of stateful SDN data planes," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 3, pp. 1701–1725, 3rd Quart., 2017.
[12] L. Dai et al., "Switching Gaussian mixture variational RNN for anomaly detection of diverse CDN websites," in *Proc. IEEE Conf. Comput. Commun.*, May 2022, pp. 300–309.
[13] Y. Li, W. Chen, B. Chen, D. Wang, L. Tian, and M. Zhou, "Prototype-oriented unsupervised anomaly detection for multivariate time series," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2023, pp. 1–18.
[14] J. Chung, K. Kastner, L. Dinh, K. Goel, A. Courville, and Y. Bengio, "A recurrent latent variable model for sequential data," in *Proc. Annu. Conf. Neural Inf. Process. Syst. (NeurIPS)*, Jan. 2015, pp. 1–9.
[15] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Disc. Data Min.*, 2019, pp. 2828–2837.
[16] L. Dai et al., "SDFVAE: Static and dynamic factorized VAE for anomaly detection of multivariate CDN KPIS," in *Proc. Web Conf.*, 2021, pp. 3076–3086.
[17] S. Tuli, G. Casale, and N. R. Jennings, "TranAD: Deep transformer networks for anomaly detection in multivariate time series data," *Proc. VLDB Endow.*, vol. 15, no. 6, pp. 1201–1214, Feb. 2022.
[18] A. Aldribi, I. Traoré, B. Moa, and O. Nwamuo, "Hypervisor-based cloud intrusion detection through online multivariate statistical change tracking," *Comput. Secur.*, vol. 88, Jan. 2020, Art. no. 101646.
[19] D. Park, Y. Hoshi, and C. C. Kemp, "A multimodal anomaly detector for robot-assisted feeding using an LSTM-based variational autoencoder," *IEEE Robot. Autom. Lett.*, vol. 3, no. 3, pp. 1544–1551, Jul. 2018.
[20] N. Dilokthanakul et al., "Deep unsupervised clustering with Gaussian mixture variational autoencoders," 2016, *arXiv:1611.02648*.
[21] J. Xu, H. Wu, J. Wang, and M. Long, "Anomaly transformer: Time series anomaly detection with association discrepancy," in *Proc. Int. Conf. Learn. Represent.*, 2021, pp. 1–20.
[22] Y. Yang, C. Zhang, T. Zhou, Q. Wen, and L. Sun, "DCdetector: Dual attention contrastive representation learning for time series anomaly detection," in *Proc. 29th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (KDD)*, 2023, pp. 3033–3045.
[23] Can. Inst. for Cybersecurity. (2018).*CSE-CIC-IDS2018 on AWS*. [Online]. Available: https://www.unb.ca/cic/datasets/ids-2018.html
[24] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2013, pp. 1–14.
[25] X. Li, Y. Chen, Z. Lin, X. Wang, and J. H. Chen, "Automatic policy generation for inter-service access control of microservices," in *Proc. 30th USENIX Secur. Symp.*, Jan. 2021, pp. 3971–3988.
[26] Weaveworks.*Sock Shop: A Microservice Demo Application*. Accessed: Mar. 1, 2024. [Online]. Available: https://github.com/microservices-demo/microservices-demo
[27] Y. Su et al., "Detecting outlier machine instances through Gaussian mixture variational autoencoder with one dimensional CNN," *IEEE Trans. Comput.*, vol. 71, no. 4, pp. 892–905, Apr. 2022.
[28] A. Vaswani et al., "Attention is all you need," in *Proc. NIPS*, 2017, pp. 1–15.
[29] E. Jang, S. Gu, and B. Poole, "Categorical reparametrization with gumble-softmax," in *Proc. Int. Conf. Learn. Represent.*, 2017, pp. 1–13.
[30] A. Siffer, P.-A. Fouque, A. Termier, and C. Largouet, "Anomaly detection in streams with extreme value theory," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2017, pp. 1067–1075.
[31] H. Xu et al., "Unsupervised anomaly detection via variational auto-encoder for seasonal KPIs in Web applications," in *Proc. World Wide Web Conf.*, 2018, pp. 187–196.
[32] D. Firestone, "VFP: A virtual switch platform for host sdn in the public cloud," in *Proc. 14th USENIX Symp. Networked Syst. Design Implement. (NSDI)*, Mar. 2017, pp. 315–328.
[33] Linux Found. *Open Vswitch*. Accessed: Feb. 15, 2025. [Online]. Available: http://www.openvswitch.org/
[34] M. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Demonstrations*, 2016, pp. 1135–1144.
[35] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.* Red Hook, NY, USA, 2017, pp. 4768–4777.

Zixuan Ma received the B.S. degree in computer science and technology from Nanjing University of Information Science and Technology in 2019. He is currently pursuing the dual Ph.D. degree with the Institute of Information Engineering (IIE), Chinese Academy of Sciences (CAS), and the University of Chinese Academy of Sciences (UCAS). His research interests include cloud network security, intent-based networks, software-defined networking, and micro-segmentation.

Chen Li received the Ph.D. degree in computer system architecture from the Institute of Information Engineering (IIE), Chinese Academy of Sciences (CAS), in 2023. She is currently an Engineer with IIE, CAS. Her current research interests include cloud data center security.

Kun Zhang received the Ph.D. degree in cyberspace security from the Institute of Information Engineering (IIE), Chinese Academy of Sciences (CAS), in 2024. She is currently a Senior Engineer with IIE, CAS. Her research interests include operating system and virtualization security.

Bibo Tu received the Ph.D. degree in computer system architecture from the Institute of Computing Technology (ICT), Chinese Academy of Sciences (CAS), in 2009. He is currently a Researcher with IIE, CAS, and a Professor with the School of Cyber Security, University of Chinese Academy of Sciences (UCAS). His current research interests include cloud data center security, trusted computing, and software defined security.